8. Technology Stack and Design Decisions

The Local Service Connect system will use Python, Flask, SQLite, HTML, and CSS. I selected these technologies based on my current knowledge and the requirements of the project. They will help me develop the web application, store information, and suggest suitable service categories based on the problems described by users.

8.1 Programming Language

Python will be the main programming language for this project. It will handle the application logic, process service requests, connect to the database, and support the text classification feature. I selected Python because I have experience with it, and it provides libraries that can help me process and classify text descriptions.

8.2 Web Framework

Flask will be used to develop the web application. It will connect the frontend pages with the Python backend and handle user actions, such as submitting a service request and checking its status. I selected Flask because it is simple to understand and suitable for a small project. Django was another option, but Flask provides the basic features I need without adding unnecessary complexity.

8.3 Frontend Technologies

HTML and CSS will be used to create the website pages. HTML will provide the form fields, buttons, navigation links, and other page elements. CSS will control the layout, colors, spacing, and appearance of the website. The service request form I designed allows users to describe their problems, select a service category, and enter equipment details and their service area. I chose HTML and CSS because they are enough to build the planned interface.

8.4 Database

SQLite will be used to store user details, provider information, service categories, service requests, classification results, status updates, and feedback. These records are related to each other, so a relational database is suitable for the project. Primary keys and foreign keys will help connect the records and maintain accurate information.

I selected SQLite because it is easy to set up and does not require a separate database server. MySQL was also considered, but SQLite is a practical choice for the current project because it is a small academic application. If the system grows and needs to support more users, MySQL could be considered later.

8.5 Libraries for Text Classification

Scikit-learn will be used for the planned text classification feature. The system will use TF-IDF to convert users' problem descriptions into numerical values that a classification model can process. Logistic Regression is the planned model for suggesting a service category based on the description.

For example, if a user enters that a washing machine is making a loud noise, the system may suggest an appliance repair category. The user can review the suggestion and select the correct category before submitting the request. The classification feature will depend on a labelled dataset containing problem descriptions and their service categories. I will identify a suitable dataset and test the model before integrating it into the application.

8.6 Development Tools

Visual Studio Code will be used to write and organize the project files. Python virtual environments and pip will help manage the libraries required by the application. Git and

GitHub will be used to save the source code, track changes, and maintain the project repository. The repository will include a README file explaining the project, the required libraries, database setup, and steps to run the application.

8.7 Hosting and APIs

During development, the application will run locally using Flask at http://localhost:5000. This address will allow me to test the website on my computer. Cloud hosting is not required for the initial version, but it could be considered if the application needs to be available online.

The application will use Flask routes to connect the frontend, backend, database, and classification feature. External APIs and paid services are not planned for the first version. Provider matching will use information stored in the database, including service categories and service areas. This approach will keep the system simple and reduce its dependence on external services.

8.8 Overall Design Decisions

The main goal is to build a working application using technologies I can understand and manage. Flask will handle the website functions, SQLite will store the information, and HTML and CSS will provide the user interface. Python and scikit-learn will support the category suggestion feature. These choices will help me develop and test the main functions, including submitting service requests, suggesting categories, finding suitable providers, and tracking request status. If the project requires more features in the future, the database and hosting options can be changed as needed.